Software Testing

ASSIGNMENT 2

CO2: Test case design strategies, black-box testing, random testing, equivalence class partitioning, boundary value analysis, other black-box techniques, white-box testing, test adequacy criteria, coverage, control flow graphs, path testing.

Assignment Title: Design, Evaluate and Defend a Test Suite for a Courier Shipping-Charge Calculator (Total Marks - 50)

Duration:1week

Scenario — Courier Shipping-Charge Calculator (SwiftShip)

A courier company is launching SwiftShip, an online service that calculates the shipping charge for a parcel and tracks it until delivery. More than 40,000 parcels are booked every day. An undercharge causes revenue loss; an overcharge causes customer complaints and refunds.

You are the Test Designer. You must design test cases, evaluate their effectiveness and defend the final test suite that will be used before every release.

Shipping-charge rules (specification extract):

Rule	Specification
S1	Weight (kg, up to one decimal place) must be greater than 0 and not more than 50. Otherwise the booking is rejected as INVALID.
S2	Charge is calculated for every started kilogram: Zone A 40/kg, Zone B 60/kg, Zone C 90/kg. Any other zone is INVALID.
S3	Express delivery adds 50 % to the charge. Express is allowed only

	up to 30 kg; above 30 kg the request is NOT_ALLOWED.
S4	**Customers who booked 10 or more parcels in the current month receive a 10 % discount on the final charge.**

Shipping-charge module – Pseudocode under test:

```
function shipCharge(weight, zone, express, parcels)
 1  if weight <= 0 or weight > 50 then
 2    return INVALID
 3  rate = 40
 4  if zone == "B" then
 5    rate = 60
 6  else if zone == "C" then
 7    rate = 90
 8  charge = 0
 9  w = weight
10  while w > 0 do
11      charge = charge + rate
12    w = w - 1
13  if express then
14    if weight > 30 then
15          return NOT_ALLOWED
16      charge = charge * 1.5
17  if parcels >= 10 then
18      charge = charge * 0.9
19  return charge
```

Parcel tracking states:

BOOKED PICKED_UP IN_TRANSIT OUT_FOR_DELIVERY DELIVERED. A parcel can move from OUT_FOR_DELIVERY to FAILED_ATTEMPT, and from FAILED_ATTEMPT back to OUT_FOR_DELIVERY (maximum two retries) or to RETURNED. A booking can be CANCELLED only before PICKED_UP.

Design Challenge:

SwiftShip will add a fuel surcharge and weekend delivery, and the shipping-charge code will be modified.

Re-evaluate your test suite and determine whether it should be retained, modified or redesigned.

TASK 1 – Select Test Case Design Strategies 5 Marks

CO2 | Bloom's Level: K5 – Evaluate

SwiftShip has a calculation module (shipCharge) and a tracking module.

Which test design strategy will find the most important defects in each module with the least effort?

Your answer must include:

- **The difference between black-box and white-box test design strategies**
- **The strategy you select for the shipping-charge rules, for the pseudocode and for the tracking states**
- **The information each strategy needs (specification, code, state model)**
- **The kinds of defects each strategy is likely and unlikely to reveal**
- **A justification for combining strategies rather than using only one**

You are expected to select and justify strategies independently for this system.

Evidence Expected

A justified selection of test design strategies for each part of SwiftShip, with the expected strengths and weaknesses of each.

TASK 2 – Evaluate Random Testing 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

A team member suggests generating 1,000 random weights and zones instead of designing test cases.

Can random testing replace systematic test design for shipCharge?

Discuss:

- **How random testing would be carried out for shipCharge**
- **The probability that random inputs hit important values such as 30.0, 30.1 and 50.0**
- **The problem of deciding the expected result (test oracle) for random inputs**

Your judgement must be supported by reasoning, not opinion.

Evidence Expected

A reasoned evaluation of the usefulness and limits of random testing for shipCharge.

TASK 3 – Design and Evaluate Equivalence Classes 6 Marks

CO2 | Bloom's Level: K5 – Evaluate

Using only the specification S1–S4, apply Equivalence Class Partitioning.

What is the smallest set of test cases that represents every valid and invalid behaviour of the shipping-charge rules?

Discuss:

- **Valid and invalid equivalence classes for weight, zone, express and parcels**
- **How you identified each class from the specification**

- **Test cases covering all valid classes with as few tests as possible**
- **Test cases covering each invalid class one at a time, and why**
- **Expected results calculated from the specification**
- **Which of your test cases would fail against the pseudocode, and what that reveals**

Simply listing classes will not receive full credit. You are expected to evaluate the effectiveness of your classes.

Evidence Expected

A complete equivalence class table, derived test cases with expected results, and an evaluation of the defects they expose.

TASK 4 – Design and Evaluate Boundary Value Tests 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Defects in charge calculations often occur at the edges of ranges.

Which boundaries in SwiftShip are most likely to hide defects, and which form of BVA is worth its cost?

Discuss:

- **Boundary values for weight (0, 0.1, 30, 30.1, 50, 50.1), the started-kilogram rule and parcels (9, 10)**
- **Normal BVA, robust BVA and worst-case BVA, and the number of test cases each produces**
- **Test cases with expected results for the boundaries you select**
- **Which form of BVA you recommend for shipCharge, and why**

You must justify the choice of BVA form against the number of test cases and the risk.

Evidence Expected

A justified boundary value test design with expected results and a defended choice of BVA form.

TASK 5 – Apply Other Black-Box Techniques 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

Express, weight and discount rules interact with each other.

Do ECP and BVA miss defects caused by combinations of rules?

Discuss:

- **A cause–effect graph or decision table for rules S3 and S4**
- **Test cases derived from the decision table**
- **Combinations that ECP/BVA did not cover, and whether they matter**

You are expected to compare the results with your ECP/BVA test set.

Evidence Expected

A decision table or cause–effect graph with derived tests and a comparison with ECP/BVA coverage.

TASK 6 – Design and Evaluate State-Transition Tests 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

The tracking module behaves differently depending on the current state of a parcel.

Which invalid transitions could cost the company money or reputation if they were allowed?

Discuss:

- **A state-transition diagram and table for parcel tracking**
- **Test cases for all valid transitions and selected invalid transitions**

Tests for the "maximum two retries" rule and the cancellation rule

You are expected to justify which invalid transitions you selected.

Evidence Expected

A state-transition model with justified valid and invalid transition tests.

TASK 7 – Construct the Control Flow Graph 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Using the pseudocode of shipCharge, construct the control flow graph.

Does the structure of the code match the structure of the specification?

Discuss:

- **Nodes and edges, with line numbers mapped to nodes**
- **Decision nodes, the loop, and the compound condition at line 1**
- **Differences between the code structure and the rules S1–S4 (for example, invalid zones)**

The graph must be derived from the code, with the mapping clearly shown.

Evidence Expected

A correct control flow graph with node–line mapping and an evaluation of mismatches between code and specification.

TASK 8 – Evaluate Path Testing and Cyclomatic Complexity 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Path testing aims to exercise independent paths through the code.

How many tests are needed to exercise the logic of shipCharge, and is that number enough?

Discuss:

- **Cyclomatic complexity calculated by at least two methods**
- **A set of independent (basis) paths and test data for each path**
- **Infeasible paths, if any, and why**
- **How the loop affects path testing, and how you would test it (zero, one, many iterations)**

You must show the derivation. Stating only the final number will not receive full credit.

Evidence Expected

A derived cyclomatic complexity, basis paths with test data, and an evaluation of loop testing for shipCharge.

TASK 9 – Judge Test Adequacy and Coverage 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

Management wants a single coverage number to decide whether testing is "enough".

Which coverage criterion is adequate for a module that calculates money for 40,000 parcels a day?

Discuss:

- **Statement, branch, condition and multiple-condition coverage for shipCharge**
- **The coverage achieved by your path tests from Task 8**
- **The additional tests needed to reach the next stronger criterion**
- **Whether 100 % coverage would have found the invalid-zone defect**

Your recommended criterion must be defended against cost and risk.

Evidence Expected

A justified adequacy criterion for shipCharge supported by coverage analysis of your own tests.

TASK 10 – Compare Black-Box and White-Box Effectiveness 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

You now have black-box tests (Tasks 3–6) and white-box tests (Tasks 7–9).

Which approach actually found more of the important defects in shipCharge?

Discuss:

- **The defects found by black-box tests only, white-box tests only, and both**
- **Reasons why each approach missed what it missed**
- **Overlapping tests that can be removed without losing defect-finding power**
- **Whether this result would hold for other modules**

You are expected to interpret the comparison, not just count tests.

Evidence Expected

A reasoned comparison of black-box and white-box effectiveness for shipCharge, supported by your own test results.

TASK 11 – Recommend and Defend the Final Test Suite 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Based on your analysis, recommend the final regression test suite for SwiftShip.

Discuss:

- **The test cases you include and the technique each comes from**

Why this suite is better than the alternatives

The coverage and the rule combinations it achieves

The number of test cases and the effort needed to run them

Known gaps and risks that remain

Your recommendation must be based on evidence from your own test design. The largest suite is not automatically the best.

Evidence Expected

A technically justified final test suite supported by black-box, white-box and coverage evidence.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Test Suite 8 Marks

CO2 | Bloom's Level: K5 – Evaluate

Changed Requirement

SwiftShip will add a fuel surcharge of 8 % on Zone C parcels and a weekend delivery option that adds 50 per parcel.

The developers have changed shipCharge by adding two new decisions after line 16.

The suite must still run within 30 minutes before every release.

Re-evaluate your original solution in response to these changed conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Discuss:

- **What has changed in the specification and the code**
- **Which equivalence classes, boundaries and decision-table rules are affected**
- **How the control flow graph and cyclomatic complexity change**
- **Why the 30-minute limit matters**

Evidence Expected

A clear analysis of how the changes affect the original test design.

B. Evaluation of the Original Suite – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Evaluate whether your final test suite remains adequate.

Discuss:

- **Which existing tests are still valid, need new expected results, or are obsolete**
- **Which new paths and rule combinations are not covered**
- **Whether the coverage criterion can still be met**

Evidence Expected

A technically supported evaluation of the original suite under the new conditions.

C. Adaptation or Redesign – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Decide whether to:

Retain, modify or redesign the test suite.

If you modify or redesign it:

- **Describe the new and changed test cases**

Explain which techniques you used to derive them

Show the effect on coverage and the number of test cases

If you retain the original suite, provide sufficient justification for why no change is necessary.

Evidence Expected

An appropriate test-design response to the changed requirement with clear reasoning.

D. Evidence-Based Defence – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Defend your final test suite using evidence.

Use:

Equivalence classes and boundaries

Decision-table combinations

Cyclomatic complexity and basis paths

Coverage achieved

Execution time limit

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final test suite integrating test design evidence and the changed constraints.

SOFTWARE TESTING (CSA37)

ASSIGNMENT 2 – CO2

Assignment Title: Design, Evaluate and Defend a Test Suite for a Courier Shipping-Charge Calculator

Scenario: SwiftShip Courier Shipping-Charge Calculator

Nature of Assignment: Individual

Total Marks: 50

Note: The answers below follow the task order in the question paper. They use the given specification and pseudocode. Where a calculation requires an assumption, the assumption is stated.

Task 1 – Select Test Case Design Strategies (5 Marks)

Choosing Strategies for SwiftShip

Black-box testing designs tests from the specification without looking at the program code. It checks whether the system gives the correct result for given inputs. White-box testing designs tests from the internal code structure, such as statements, branches, conditions and paths. Black-box testing is useful for finding missing or wrongly implemented business rules. White-box testing is useful for finding code paths and decisions that have not been exercised.

SwiftShip part	Selected strategy	Information needed	Likely strengths and limitations
Shipping-charge rules	Equivalence Class Partitioning (ECP), Boundary Value Analysis (BVA), and decision-table testing	Specification S1–S4; valid ranges and rule combinations	Finds wrong limits, invalid inputs, rate and discount errors. It may miss code paths not represented in the specification.
shipCharge pseudocode	White-box statement, branch, condition and basis-path testing	Source pseudocode and control-flow graph	Finds unexecuted branches, loop errors and missing handling in code. It may not reveal a requirement that was never written.
Parcel tracking states	State-transition testing	State model, permitted	Finds illegal state changes,

		transitions, retry and cancellation rules	retry-limit errors and cancellation after pickup. It may miss calculation defects unrelated to states.

For the shipping-charge rules, ECP groups inputs that should behave alike. For example, weights from 0.1 kg to 50.0 kg form the valid weight class, while zero and weights above 50 kg are invalid classes. BVA then checks values at and around important limits. A decision table is needed because express delivery and the monthly-parcel discount can occur together.

For the pseudocode, white-box testing is appropriate because the code contains a compound weight condition, zone decisions, a loop, an express decision and a discount decision. The tester can build a control-flow graph and choose tests that exercise each independent branch and path.

For tracking, a state-transition model is the best choice because a parcel's permitted next state depends on its current state. A simple input/output test may not detect a parcel being cancelled after pickup or being returned after more than two failed delivery retries.

No single technique is sufficient. Black-box tests check the user's rules, white-box tests check the implementation structure, and state-transition tests check the parcel lifecycle. Combining them gives stronger defect detection with a manageable number of tests.

Task 2 – Evaluate Random Testing (2 Marks)

Can Random Testing Replace Systematic Testing?

No. Random testing can supplement systematic testing, but it should not replace ECP, BVA or decision-table testing. A random test generator would select weights, zones, express values and parcel counts from chosen input ranges, run shipCharge, and compare the result with an independently calculated expected result.

If weights are generated uniformly to one decimal place from 0.0 to 50.0, there are 501 possible values. The probability of selecting any one exact value, such as 30.0, 30.1 or 50.0, is 1/501 (about 0.20%) per test. With 1,000 independent tests, each exact value is likely to appear about twice on average, but it is not guaranteed. If weights are generated as continuous decimals, the probability of hitting an exact value is effectively zero.

A further difficulty is the test oracle: the expected result must be known. A random input is useful only if the tester can calculate the correct charge from the specification, including started-kilogram rounding, zone rate, express surcharge and discount. If the oracle repeats the same coding mistake, the test may incorrectly pass.

Therefore, systematic tests must deliberately include 30.0, 30.1, 50.0, invalid zones and the 9/10-parcel boundary. Random testing can then be added to explore unexpected combinations and unusual inputs.

Task 3 – Design and Evaluate Equivalence Classes (6 Marks)

Equivalence Class Partitioning

ECP divides the input domain into groups expected to behave in the same way. The classes below are derived only from rules S1–S4.

Input	Valid equivalence classes	Invalid equivalence classes
Weight	$0 < \text{weight} \leq 50$ kg, in steps of 0.1 kg	weight ≤ 0; weight > 50 kg; values not representable to one decimal place (if the interface accepts them)
Zone	A, B or C	Any value other than A, B or C
Express	False; True with weight ≤ 30 kg	True with weight > 30 kg
Parcels booked this month	0–9; 10 or more	Negative parcel count (if the input interface permits it)

The smallest practical valid set below covers each valid class at least once and also combines express and discount behaviour. Expected charge uses the started-kilogram

rule: charge = ceiling(weight) × zone rate; multiply by 1.5 for express and by 0.9 when parcels ≥ 10.

Test	Weight	Zone	Express	Parcels	Expected result
V1	1.0	A	No	0	₹40
V2	2.1	B	No	9	₹180 (3 × ₹60)
V3	1.1	C	Yes	10	₹267.30 (2 × ₹90 × 1.5 × 0.9)
V4	30.0	A	Yes	9	₹1,800 (30 × ₹40 × 1.5)
V5	50.0	C	No	10	₹4,050 (50 × ₹90 × 0.9)

Invalid classes should be tested one at a time while the other inputs remain valid. This isolates the reason for rejection.

Test	Weight	Zone	Express	Parcels	Expected result
I1	0.0	A	No	0	INVALID
I2	50.1	A	No	0	INVALID
I3	1.0	D	No	0	INVALID
I4	30.1	A	Yes	0	NOT_ALLOWED
I5	1.0	A	No	-1	Invalid input / rejected, if negative parcel counts are disallowed

Evaluation against the pseudocode: I1 and I2 should be rejected by line 1. I3 is expected to be INVALID under S2, but the pseudocode leaves the default rate at ₹40 for every zone other than B or C. Therefore I3 would incorrectly return a charge. I4 should return NOT_ALLOWED at line 15. I5 is not explicitly defined in S4; the specification says customers with 10 or more parcels receive a discount but does not state how negative counts are handled. It should be clarified rather than treated as a confirmed specification rule.

The valid tests show that the classes work across all three zones, both express choices, both discount groups and the upper valid weight. The invalid tests isolate each specified

rejection condition. Additional combinations may still be needed for stronger interaction coverage.

Task 4 – Design and Evaluate Boundary Value Tests (4 Marks)

Boundary Value Analysis

The important boundaries are weight just above zero, the 30 kg express limit, the 50 kg maximum, each started-kilogram step, and the discount threshold between 9 and 10 parcels.

Boundary	Values to test	Expected behaviour
Minimum weight	0.0, 0.1, 0.2	0.0 = INVALID; 0.1 and 0.2 are valid
Express limit	29.9, 30.0, 30.1	Express allowed at 29.9 and 30.0; 30.1 = NOT_ALLOWED
Maximum weight	49.9, 50.0, 50.1	49.9 and 50.0 valid; 50.1 = INVALID
Started kilogram	1.0, 1.1, 2.0, 2.1	Charges use 1, 2, 2 and 3 started kilograms respectively
Discount threshold	9, 10, 11 parcels	No discount at 9; 10 and 11 receive 10% discount

Normal BVA generally tests the minimum, just above minimum, a nominal value, just below maximum and maximum for each input. Robust BVA also tests just outside the valid range. For n independent numeric variables, the common counts are 4n+1 for normal BVA and 6n+1 for robust BVA. Worst-case BVA tests all combinations of the selected boundary values; with five values for each of n variables it can require 5^n tests (or more when robust values are included).

For this module, full worst-case testing would create many combinations and is not cost-effective. I recommend robust BVA focused on the high-risk boundaries, plus explicit started-kilogram and parcel-count tests. The money calculation and rejection limits are high risk, so testing just outside the valid limits is worth the extra effort.

Test	Weight	Zone	Express	Parcels	Expected result
B1	0.0	A	No	9	INVALID
B2	0.1	A	No	9	₹40
B3	29.9	A	Yes	9	₹1,800 (30 × ₹40

					× 1.5)
B4	30.0	A	Yes	9	₹1,800
B5	30.1	A	Yes	9	NOT_ALLOWED
B6	49.9	B	No	10	₹2,700 (50 × ₹60 × 0.9)
B7	50.0	C	No	9	₹4,500
B8	50.1	C	No	9	INVALID
B9	1.1	A	No	9	₹80 (2 × ₹40)
B10	1.1	A	No	10	₹72 (₹80 × 0.9)

The test set concentrates on boundaries that can change acceptance or price. It is more useful than testing many ordinary middle values because off-by-one and rounding mistakes are especially likely at these points.

Task 5 – Apply Other Black-Box Techniques (3 Marks)

Decision Table for Express and Discount Rules

ECP and BVA test individual classes and edges, but they may not test every interaction between express delivery and the discount. A decision table makes these combinations explicit.

Condition / action	Rule 1	Rule 2	Rule 3	Rule 4
Express requested?	No	No	Yes	Yes
Weight above 30 kg?	–	–	No	Yes
Parcels ≥ 10?	No	Yes	No	Either
Express surcharge	No	No	Add 50%	Not applicable
Discount	No	Apply 10%	No	Not applicable
Result	Base charge	Base × 0.9	Base × 1.5	NOT_ALLOWED

The decision table assumes that an express request over 30 kg is rejected before a discount is applied, as stated by S3. The order of surcharge and discount does not change the mathematical result when both are permitted because multiplication by 1.5 and 0.9 can be performed in either order.

Test	Weight	Zone	Express	Parcels	Expected result
D1	2.0	A	No	9	₹80
D2	2.0	A	No	10	₹72

D3	2.0	A	Yes	9	₹120
D4	2.0	A	Yes	10	₹108
D5	30.1	B	Yes	10	NOT_ALLOWED

The first four tests cover the four meaningful combinations of express and discount. D5 confirms that the express weight restriction takes priority even when a discount is available. ECP/BVA tests may include some of these cases, but the decision table proves that all important combinations have been deliberately checked.

Task 6 – Design and Evaluate State-Transition Tests (3 Marks)

Parcel Tracking State Model

The valid tracking route is BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED. From OUT_FOR_DELIVERY, a failed attempt may move to FAILED_ATTEMPT. From FAILED_ATTEMPT, the parcel may return to OUT_FOR_DELIVERY for a maximum of two retries, or move to RETURNED. Cancellation is allowed only before PICKED_UP.

State-transition diagram (text form):

BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED
↓ failed attempt
FAILED_ATTEMPT
↙ ↘
OUT_FOR_DELIVERY RETURNED
(maximum 2 retries)
BOOKED → CANCELLED (only before PICKED_UP)

Current state	Event	Next state	Validity
BOOKED	Pickup confirmed	PICKED_UP	Valid
BOOKED	Cancel booking	CANCELLED	Valid
PICKED_UP	Parcel enters transit	IN_TRANSIT	Valid
IN_TRANSIT	Courier starts final delivery	OUT_FOR_DELIVERY	Valid
OUT_FOR_DELIVERY	Delivery successful	DELIVERED	Valid
OUT_FOR_DELIVERY	Delivery failed	FAILED_ATTEMPT	Valid
FAILED_ATTEMPT	Retry within allowed limit	OUT_FOR_DELIVERY	Valid, maximum two retries

FAILED_ATTEMPT	Retry limit reached / return decision	RETURNED	Valid
PICKED_UP	Cancel booking	CANCELLED	Invalid
DELIVERED	Move back to IN_TRANSIT	IN_TRANSIT	Invalid
CANCELLED	Pickup parcel	PICKED_UP	Invalid

Important tests are: complete a successful delivery; cancel while BOOKED; attempt cancellation after PICKED_UP; record one failed attempt and retry; record two retries and confirm that no third retry is allowed; and move to RETURNED after the retry limit. Invalid transitions are selected because they could cause lost parcels, incorrect customer updates, duplicate delivery work or refunds for parcels already collected.

Task 7 – Construct the Control Flow Graph (4 Marks)

Control-Flow Graph of shipCharge

The following node mapping groups sequential statements and keeps each decision as a separate node.

Node	Pseudocode lines	Meaning
N1	1–2	Check invalid weight; return INVALID if true
N2	3–7	Set default rate and select B or C rate
N3	8–9	Initialise charge and working weight
N4	10	Loop condition: w > 0?
N5	11–12	Add one rate and subtract 1 from w
N6	13	Express requested?
N7	14–15	If weight > 30, return NOT_ALLOWED
N8	16	Apply 50% express surcharge
N9	17	Parcels ≥ 10?
N10	18	Apply 10% discount
N11	19	Return charge

Edges: Start → N1. N1 true → INVALID/End; N1 false → N2. N2 → N3 → N4. N4 true → N5 → N4; N4 false → N6. N6 false → N9; N6 true → N7. N7 true →

NOT_ALLOWED/End; N7 false → N8 → N9. N9 true → N10 → N11; N9 false → N11. N11 → End.

The compound condition at line 1 is (weight ≤ 0) OR (weight > 50). Both parts contribute to invalid-weight detection. The loop repeats once for every started kilogram because w is reduced by 1 until it is no longer greater than zero.

The code does not fully match the specification. S2 says every zone other than A, B or C is INVALID. The code sets the default rate to 40 and changes it only for B and C; therefore an invalid zone is silently treated as Zone A. The graph has no invalid-zone decision or return path. This is a specification-to-code mismatch and is a defect even if every existing code branch is covered.

Task 8 – Evaluate Path Testing and Cyclomatic Complexity (4 Marks)

Cyclomatic Complexity and Basis Paths

At the decision-node level, the pseudocode has seven decisions: the compound invalid-weight IF, zone B, zone C, the WHILE loop, express requested, weight > 30, and parcels ≥ 10. Therefore, V(G) = number of decision nodes + 1 = 7 + 1 = 8.

Using the graph formula, V(G) = E − N + 2P. Counting the simplified graph with a single common exit gives 22 edges (E = 22), 16 nodes (N = 16), and one connected component (P = 1). Therefore, V(G) = 22 − 16 + 2(1) = 8. This agrees with the decision-node method.

A basis set contains eight linearly independent paths. Representative paths and data are shown below. Paths are representative at the decision level; loop iteration counts are tested separately.

Path	Route through decisions	Test data	Purpose / outcome
P1	Invalid weight = true	weight 0, zone A, no express, 0 parcels	INVALID
P2	Valid; zone A; loop; no express; no discount	1.0, A, false, 0	Base Zone A result
P3	Valid; zone B branch	1.0, B, false, 0	Zone B rate

P4	Valid; zone C branch	1.0, C, false, 0	Zone C rate
P5	Valid; express false; discount true	1.0, A, false, 10	Discount branch
P6	Valid; express true; weight > 30	30.1, A, true, 0	NOT_ALLOWED branch
P7	Valid; express true; weight ≤ 30; no discount	2.0, A, true, 0	Express surcharge
P8	Valid; express true; weight ≤ 30; discount true	2.0, A, true, 10	Combined surcharge and discount

Some paths cannot be executed as a complete route because an earlier return prevents reaching later decisions. For example, an invalid weight exits before the zone decision. These are infeasible combinations of branch outcomes, not missing test data.

The loop needs separate tests for zero, one and many iterations. A zero-iteration loop is not reachable for a valid weight because S1 requires weight > 0; it can only occur if an invalid weight bypassed the first check, which should not happen. One iteration can be tested with 0.1 or 1.0 kg. Multiple iterations can be tested with 2.1 kg or 50.0 kg. A decimal such as 2.1 should execute three iterations because the rule charges for every started kilogram.

Cyclomatic complexity gives the number of independent paths in the decision structure, not every possible loop repetition. It is a useful minimum basis, but it does not prove that all weight values or all business rules are correct.

Task 9 – Judge Test Adequacy and Coverage (3 Marks)

Recommended Coverage Criterion

Statement coverage checks whether every executable statement runs. Branch coverage checks both outcomes of each decision. Condition coverage checks whether each individual Boolean condition becomes true and false. Multiple-condition coverage checks combinations of the conditions within compound decisions.

Criterion	What it checks in shipCharge	Limitation
Statement	Each statement executes at least once	May miss the false side of a decision
Branch	True and false outcomes of each	Does not necessarily test every

	decision execute	combination of atomic conditions
Condition	Each atomic condition is true and false	May not cover every combined decision outcome
Multiple-condition	All combinations of conditions are exercised	Can create many tests; often costly

The basis tests in Task 8 are intended to cover each reachable branch in the code: invalid and valid weight, zone B/C and default zone, loop exit and loop body, express true/false, express weight permitted/not permitted, and discount true/false. However, branch coverage alone does not test every combination of the two atomic conditions at line 1.

To strengthen the compound condition, test (weight ≤ 0, weight > 50) combinations that demonstrate each condition's independent effect: weight 0 (first true), weight 50.1 (second true), and a valid value such as 1.0 (both false). This is a practical modified condition/decision check for the OR expression.

I recommend 100% branch coverage plus condition coverage for the high-risk charge decisions, together with explicit ECP/BVA and decision-table tests. Full multiple-condition coverage can be considered for the compound weight check, but it should not replace business-rule tests. Even 100% code coverage would not find the invalid-zone defect if tests never supply an invalid zone: the code has no invalid-zone branch to cover. A specification-derived test is required to expose it.

Task 10 – Compare Black-Box and White-Box Effectiveness (4 Marks)

Comparison of Defect-Finding Power

Defect / risk	Black-box tests	White-box tests	Reason
Invalid zone accepted as Zone A	Finds it with zone D test	May not find it because no invalid-zone branch exists	The missing rule is visible from S2, not from existing code structure
Wrong started-kilogram calculation	Finds it with 1.1 and 2.1 kg cases	Finds loop iteration issues if designed carefully	Both methods can expose the calculation error
Express limit error at 30.1 kg	Finds it from S3 boundary	Finds the weight > 30 branch	Both can expose it
Discount not applied at 10	Finds it with 9/10 boundary	Finds discount branch if	Both can expose it

parcels		both outcomes tested	
Untested return or loop exit path	May miss it if inputs do not target it	Targets it through branch/path coverage	White-box advantage

Black-box testing is stronger for finding defects caused by a missing or misunderstood specification rule, especially the invalid-zone rule. White-box testing is stronger for showing whether every decision, loop and return path in the actual pseudocode has been exercised. The two approaches overlap on the express and discount branches, so duplicate tests may be reduced only when one test genuinely covers the same input condition, expected result and code branch.

The result may differ in another module. A user-interface module may benefit more from black-box usability and requirements testing, while a complex algorithm may need more white-box path testing. For SwiftShip, both are required because incorrect money calculations and unhandled input classes are important risks.

Task 11 – Recommend and Defend the Final Test Suite (4 Marks)

Final Regression Test Suite

I recommend a compact regression suite that combines ECP, robust BVA, decision-table, state-transition and white-box tests. The goal is not to select the largest number of tests; it is to cover each important rule, each high-risk boundary, each meaningful rule combination and each important code branch.

Group	Included tests	Technique	Reason
Input validity	0.0, 0.1, 50.0, 50.1; valid and invalid zones	ECP + robust BVA	Covers acceptance limits and invalid zone
Started kg	1.0, 1.1, 2.0, 2.1	BVA	Checks rounding to started kilograms
Express	30.0, 30.1 with express on	BVA + decision table	Checks allowed limit and rejection
Discount	9 and 10 parcels	BVA	Checks discount threshold
Rule interactions	Express on/off × discount on/off	Decision table	Covers combinations
Code structure	Invalid weight, each zone, loop body/exit, express and	Branch/basis-path testing	Provides white-box adequacy evidence

	discount outcomes		
Tracking	Successful delivery, cancellation, failed attempt, two retries, third retry rejected, return	State-transition testing	Protects parcel lifecycle

The core suite should be kept small by reusing tests that cover more than one objective. For example, a 2.0 kg Zone A express test with 10 parcels covers the express branch, the discount branch and their interaction. However, it must not replace a separate test for an invalid zone or the 30.1 kg express rejection.

Before each release, the suite should run in this order: quick input and calculation checks; high-risk boundary and decision-table checks; state-transition checks; then the full branch regression. Expected results should be calculated independently from S1–S4 and reviewed, not copied from the program output.

Known gaps include unspecified handling of negative parcel counts, invalid express input formats, currency rounding policy and exact transition details after a failed delivery. These should be clarified with the product owner. The suite is preferable to random-only testing because every requirement is deliberately represented and the number of tests remains manageable.

Task 12 – Re-evaluate and Adapt the Test Suite (8 Marks)

Changed Requirements: Fuel Surcharge and Weekend Delivery

The new requirements add an 8% fuel surcharge for Zone C parcels and a fixed weekend-delivery charge of ₹50 per parcel. The original suite should be MODIFIED, not retained unchanged and not completely redesigned. The original tests remain useful for the existing rules, but expected results and rule combinations must be extended.

A. Impact Analysis (2 Marks)

Area	Original rule	New requirement	Impact
Zone C charge	Zone C rate = ₹90 per started kg	Add 8% fuel surcharge for Zone C	Every Zone C expected result changes

Weekend option	No weekend option	Add ₹50 per parcel	New input equivalence classes and decision added
Decision table	Express × discount	Express × discount × weekend × Zone C surcharge	More combinations must be checked
Control flow	Seven decision nodes at the decision level	Two new decisions after line 16	Cyclomatic complexity increases

At the code level, two new decisions are added after line 16. Assuming each is a separate decision node, the new cyclomatic complexity becomes 10 (old value 8 + 2). If the fuel surcharge is implemented as a Zone C condition and weekend delivery as a weekend Boolean condition, both true and false outcomes must be tested.

The 30-minute execution limit matters because the test suite must be small, prioritised and partly automated. Exhaustively testing every combination would be inefficient; tests should focus on new conditions and interactions while retaining high-risk regression cases.

B. Evaluation of the Original Suite (2 Marks)

The original tests for invalid weights, started kilograms, express limits, discount threshold, tracking states and invalid zones remain valid as test inputs. However, all Zone C expected charges must be recalculated to include the 8% surcharge. Tests using weekend delivery cannot be retained as-is because that input did not previously exist.

The new untested paths include Zone C fuel surcharge applied/not applied, weekend charge applied/not applied, and combinations with express and discount. The original branch coverage is no longer sufficient because two new decisions have been introduced. The coverage criterion should remain at least 100% branch coverage with condition coverage for high-risk decisions, and the new branches must be included.

C. Adaptation or Redesign (2 Marks)

I choose MODIFY. I would keep the existing ECP, BVA, decision-table, state-transition and basis-path tests, then add focused tests for the surcharge and weekend option. I

would also automate the arithmetic checks so the full regression can finish within 30 minutes.

For calculation examples, I assume the fuel surcharge is applied to the Zone C base charge, weekend delivery adds a fixed ₹50 per parcel, and the express surcharge and discount continue to apply as specified. The requirement does not explicitly state the order of these new charges or whether the 10% discount applies to surcharges. This must be confirmed before finalising the oracle.

Test	Weight	Zone	Express	Parcels	Weekend	Expected calculation
N1	1.0	C	No	1	No	₹90 × 1.08 = ₹97.20
N2	1.0	C	No	1	Yes	₹97.20 + ₹50 = ₹147.20
N3	2.0	C	Yes	10	No	₹180 × 1.08 × 1.5 × 0.9 = ₹262.44
N4	2.0	A	No	2	Yes	₹80 + (2 × ₹50) = ₹180
N5	2.0	B	No	1	No	₹120; no Zone C surcharge
N6	30.1	C	Yes	10	Yes	NOT_ALLOWED; express limit still applies

The table assumes weekend delivery is charged per parcel, so two parcels add ₹100. If the business intends a single ₹50 charge per booking, the expected results must be changed. This is a requirement clarification, not something the tester should guess.

The added tests should cover the new decision outcomes and important combinations: Zone C with and without weekend delivery, non-Zone-C weekend delivery, Zone C with express and discount, and an express request over 30 kg with weekend selected. Existing tests continue to protect the original rules.

D. Evidence-Based Defence (2 Marks)

Evidence	What it demonstrates	Decision supported
Existing ECP/BVA tests	Original weight, zone, express and parcel classes remain relevant	Retain them
New surcharge rule	Every Zone C charge changes by 8%	Add Zone C recalculation tests

Weekend option	₹50 is added per parcel and creates a new decision	Add weekend on/off and parcel-count tests
Cyclomatic complexity	Increases from 8 to 10 if two separate decisions are added	Add tests for both outcomes of each decision
30-minute limit	Large exhaustive combinations may exceed the time limit	Use risk-based, automated regression

The modified suite is the best choice because it preserves proven tests, adds the minimum tests needed for the new rules and keeps the regression time under control. Before release, the team must confirm the surcharge order, discount treatment and meaning of ₹50 per parcel. The remaining risk is ambiguity in these new business rules; a reviewed requirement and independent expected-result calculation will reduce that risk.

MARK DISTRIBUTION

Task	Marks
Task 1 – Select Test Case Design Strategies	**5**
Task 2 – Evaluate Random Testing	**2**
Task 3 – Design and Evaluate Equivalence Classes	**6**
Task 4 – Design and Evaluate Boundary Value Tests	**4**
Task 5 – Apply Other Black-Box Techniques	**3**
Task 6 – Design and Evaluate State-Transition Tests	**3**
Task 7 – Construct the Control Flow Graph	**4**
Task 8 – Evaluate Path Testing and Cyclomatic Complexity	**4**
Task 9 – Judge Test Adequacy and Coverage	**3**
Task 10 – Compare Black-Box and White-Box Effectiveness	**4**
Task 11 – Recommend and Defend the Final Test Suite	**4**
Subtotal	**42**
Task 12 – Day-4 Design Challenge	**8**
TOTAL	**50**